

Table of Contents

Unit 5: Web Development with HTML, CSS, and JavaScript	5
Student Learning Outcomes	5
Introduction	6
5.1 Web Development	6
What Is Web Development?	6
The Hook	6
Why Learn Web Development?	7
Interactive Stop-Point: Pause & Think	7
Quick Recap	7
5.2 Basic Components of Web Development	7
The Three Layers of Every Website	7
The Hook	8
Layer 1: Front-End Development	8
Layer 2: Back-End Development	8
Layer 3: Full-Stack Development	9
Interactive Stop-Point: Grab a Partner	9
Quick Recap	9
5.3 Getting Started with HTML	10
What Is HTML?	10
The Hook	10
Setting Up Your Development Environment	10
Creating Your First HTML Page: "Hello, World!"	11
Step-by-Step Walkthrough	11
Interactive Stop-Point: Pause & Think	11
Quick Recap	12
5.4 HTML Basic Structure	12
Understanding the Anatomy of an HTML Document	12
The Hook	12
The HTML Skeleton — Explained Line by Line	12
HTML Tags — The Building Blocks	13
Type 1: Paired Tags (Opening + Closing)	13
Type 2: Singular Tags (Self-Closing)	13
Interactive Stop-Point: Grab a Partner	14
Quick Recap	14
5.5 Creating Content with HTML	14

Introduction	14
5.5.1 Headings	14
The Hook.....	14
The Explanation	15
Example: Headings in a Page.....	15
Interactive Stop-Point: Pause & Think	16
Quick Recap.....	16
5.5.2 Paragraphs	16
The Explanation	16
Interactive Stop-Point: Pause & Think	16
Quick Recap.....	17
5.5.3 Links	17
The Hook.....	17
The Explanation	17
Example: A Page with Links	18
Interactive Stop-Point: Grab a Partner	18
Quick Recap.....	18
5.5.4 Images	18
The Explanation	18
Example	19
Interactive Stop-Point: Pause & Think	19
Quick Recap.....	20
5.5.5 Lists	20
The Hook.....	20
The Explanation	20
Example: Lists on a Page	21
Interactive Stop-Point: Grab a Partner	21
Quick Recap.....	22
5.5.6 Tables	22
The Hook.....	22
The Explanation	22
Example: A Student Marks Table	23
Interactive Stop-Point: Pause & Think	23
Quick Recap.....	24
5.5.7 HTML Comments.....	24
The Explanation	24
Example	25
Interactive Stop-Point: Pause & Think	25
Quick Recap.....	25

5.6 Styling with CSS	25
What Is CSS?	25
The Hook.....	26
5.6.1 Basic Structure of CSS	26
Full CSS Example in an HTML Page	27
5.6.2 Three Ways to Add CSS to HTML	27
The Hook.....	27
Method 1: Inline Styles.....	27
Method 2: Internal Styles	28
Method 3: External Stylesheet	28
Comparison Table.....	29
Interactive Stop-Point: Grab a Partner	29
Quick Recap.....	29
5.6.3 Styling Fonts, Colors, and Backgrounds	29
Styling Fonts	29
Styling Colors and Backgrounds	30
Interactive Stop-Point: Pause & Think	31
Quick Recap.....	31
5.6.4 Creating Layouts with CSS.....	31
The Hook.....	31
Method 1: Divs and Sections	32
Method 2: CSS Grid	32
Method 3: CSS Flexbox	33
Comparison: When to Use Each	34
Interactive Stop-Point: Grab a Partner	34
Quick Recap.....	35
5.7 Introduction to JavaScript	35
What Is JavaScript?.....	35
The Hook.....	35
Where Does JavaScript Go in HTML?	35
5.7.1 Variables and Data Types.....	36
The Explanation	36
Example: Variables and Alert	37
Interactive Stop-Point: Pause & Think	37
Quick Recap.....	38
5.7.2 Functions in JavaScript	38
The Hook.....	38
The Explanation	38

Example 1: Simple Function	39
Example 2: Function with Parameters	39
Example 3: Function with Multiple Parameters	40
Class Activity	40
Interactive Stop-Point: Grab a Partner	40
Quick Recap.....	41
Chapter Summary	41
Key Vocabulary.....	43
Review Questions.....	44
Practical Exercises	45

Unit 5: Web Development with HTML, CSS, and JavaScript

"The web is more a social creation than a technical one. I designed it for a social effect — to help people work together — and not as a technical toy."

— Tim Berners-Lee, inventor of the World Wide Web

Student Learning Outcomes

By the end of this chapter, you will be able to:

- Understand **JavaScript syntax** and data types.
 - Work with **variables, operators, and functions** in JavaScript.
 - Create **simple programs** using JavaScript.
 - Create **HTML forms** and style them.
 - Use JavaScript to **handle events**, operators, variables, and functions.
 - **Develop static web pages** using HTML and CSS.
 - Apply **HTML tags** appropriately to create web pages.
 - Create a **basic HTML page** from scratch.
 - Add **text, images, and links** to a web page.
 - Create **lists and tables** in HTML.
 - Apply **CSS styles** to HTML elements.
 - Work with **fonts, colors, and backgrounds** using CSS.
 - Create web pages to display data in **paragraphs and lists**.
 - Understand **CSS syntax** and its structure.
 - Create **layouts** with CSS (Grid and Flexbox).
 - **Organize images and text** effectively on a web page.
-

Introduction

Think about the last time you opened a website. Maybe it was YouTube, a news page, or your school's website. You saw headings, images, buttons, menus, and colors. You clicked a link and went to a new page. You typed in a search box and got results.

Who built all of that?

Web developers did. And the tools they used are exactly what you will learn in this chapter: **HTML**, **CSS**, and **JavaScript**.

- **HTML** is the skeleton — it gives a web page its structure.
- **CSS** is the skin and clothes — it makes a web page look good.
- **JavaScript** is the muscles — it makes a web page move and respond.

By the end of this chapter, you will build your own web pages from scratch. You will write real code that runs in a real browser. You will see your work displayed on screen.

Let's build something.

5.1 Web Development

What Is Web Development?

Definition: Web development is the process of creating websites and web applications using programming languages and tools to design, build, and maintain them.

When you type a web address into your browser and a page appears — someone built that. Web development is the skill of building those pages and the systems that power them.

The Hook

In 1994, two Stanford University students named **Sabeer Bhatia** and **Jack Smith** had a simple idea: what if you could check your email from any computer in the world, without installing any software? They built **Hotmail** — one of the first web-based email services — using web development skills.

Microsoft bought Hotmail in 1997 for **\$400 million**.

They started with the same skills you are about to learn.

Why Learn Web Development?

Reason	What It Means for You
Digital Literacy	You understand how the internet actually works — not just how to use it.
Career Opportunities	Web developers are needed by almost every company. It opens many job paths.
Problem-Solving	Building websites means solving real problems: slow pages, broken layouts, bad designs.
Creativity	You can design anything — a personal blog, a portfolio, a game.
Entrepreneurship	With web skills, you can build and launch your own online product or business.

Interactive Stop-Point: Pause & Think

Open any website right now (or think of one you use daily). Look at it carefully and ask:

1. What is the **structure** — what are the different sections? (header, content, footer?)
2. What colors and fonts do you see? Who decided those?
3. What happens when you click a button? Who made that work?

Web development answers all three questions. Keep these observations in mind as you learn.

Quick Recap

Web development is the skill of building websites using code. It combines structure (HTML), design (CSS), and interactivity (JavaScript).

5.2 Basic Components of Web Development

The Three Layers of Every Website

Every website — from a simple school page to a complex social media platform — is built from the same three layers:

The Hook

Think of a building. It has:

- A **steel frame** that holds the structure up (you never see it).
- **Walls, windows, and paint** that make it look like a building (what you see).
- **Electricity, plumbing, and lifts** that make it functional (what makes it work).

Websites are the same.

Layer 1: Front-End Development

Definition: Front-end development is everything the user **sees and interacts with** on a website.

The three front-end technologies:

Technology	Role	Analogy
HTML	Structures the content — headings, paragraphs, images, links.	The steel frame of a building.
CSS	Styles the content — colors, fonts, spacing, layout.	The walls, paint, and decoration.
JavaScript	Adds interactivity — buttons, animations, forms, games.	The electricity and lifts.

Layer 2: Back-End Development

Definition: Back-end development manages the **behind-the-scenes** part of a website — the servers, databases, and logic users never see directly.

Component	What It Does
Web Servers	Store and deliver web pages when a user types a URL.
Databases	Store data — user accounts, product details, posts.
Back-End Languages	PHP, Python, Ruby — process forms, manage logins, run business logic.

Example: When you log into Instagram, the front-end (HTML/CSS/JS) shows you the login form. The back-end checks your username and password against the database and decides whether to let you in.

Layer 3: Full-Stack Development

Definition: A full-stack developer works on **both** the front-end and the back-end. They can build a complete website from start to finish.

Example — A Login System:

Task	Who Does It
Design the login form (input boxes, button, colors)	Front-end developer
Check if username and password are correct	Back-end developer
Store user accounts in a database	Back-end developer
Show error or success message to the user	Front-end developer
Build the entire system alone	Full-stack developer

Full-stack developers are in very high demand because they can handle everything.

Did You Know? The very first website in history was created by **Tim Berners-Lee in 1991**. It was a simple page with text and links. You can still visit it today at: `http://info.cern.ch`

Interactive Stop-Point: Grab a Partner

Think of a website you both use (YouTube, WhatsApp Web, an online shop).

With your partner, answer:

1. Name **two things** that are front-end (things you see and click).
2. Name **two things** that must be back-end (data stored somewhere, logic happening behind the scenes).
3. If one person had to build the whole website alone, what skills would they need?

Quick Recap

Every website has three layers: front-end (what you see), back-end (what runs behind the scenes), and full-stack (building both).

5.3 Getting Started with HTML

What Is HTML?

Definition: HTML stands for **HyperText Markup Language**. It is the standard language used to create web pages. HTML uses tags to define the structure and content of a web page.

Think of HTML like **LEGO bricks**. Each brick (tag) has a specific shape and purpose. You stack them together to build a complete structure.

The Hook

In 1991, Tim Berners-Lee wrote the first ever HTML document at CERN (a physics research laboratory in Switzerland). He was trying to help scientists share documents over a network. He had no idea he was inventing the foundation of something that would one day be used by **5 billion people**.

His first HTML document had headings, paragraphs, and links. Those same elements are what you will write today.

Setting Up Your Development Environment

Before you write any code, you need two tools. Both are probably already on your computer.

Tool	What It Is	Examples
Text Editor	Where you write your HTML code.	Notepad (Windows), Notepad++, Visual Studio Code, Sublime Text
Web Browser	Where you view and test your HTML files.	Google Chrome, Mozilla Firefox, Microsoft Edge

That's it. You do not need to install anything special to start. Open Notepad, write code, save as `.html`, open in Chrome. Done.

Creating Your First HTML Page: "Hello, World!"

This is a tradition in programming. Every programmer's first program displays "Hello, World!" on the screen. Here is yours.

Step-by-Step Walkthrough

Step 1: Open your text editor (Notepad or any other).

Step 2: Type the following code exactly:

```
<!DOCTYPE html>
<html>
  <head>
    <title>My First Web Page</title>
  </head>
  <body>
    <h1>Welcome to My Website</h1>
    <p>This is my first web page. I am learning HTML in 9th class!</p>
  </body>
</html>
```

Step 3: Save the file.

- In Notepad: Go to **File** → **Save As**.
- In the "File name" box, type: `my_first_website.html`
- In the "Save as type" dropdown, select **All Files**.
- Click **Save**.

Step 4: Find the file on your computer. Double-click it. It will open in your web browser.

Step 5: You should see a web page with a large heading "Welcome to My Website" and a paragraph below it.

Tip: If you make any changes to the HTML file, press **Ctrl + S** to save, then press **F5** in the browser to refresh and see the updates.

Interactive Stop-Point: Pause & Think

Before reading Section 5.4, look at your HTML code and try to answer:

1. What do you think `<h1>` does?
2. What do you think `<p>` does?
3. What is inside the `<head>` section? What is inside the `<body>` section?
4. Why do you think every tag has a matching closing tag like `</h1>` and `</p>`?

Write your guesses down. You will find the answers in the next section.

Quick Recap

To build a web page: write HTML code in a text editor, save it with a `.html` extension, and open it in a browser. No special software needed.

5.4 HTML Basic Structure

Understanding the Anatomy of an HTML Document

Every HTML document follows the same basic skeleton. It does not matter if the page is a simple "Hello World" or a complex news website — the skeleton is always the same.

The Hook

A doctor can look at an X-ray and instantly understand the structure of a body. They know where the skull is, where the spine is, where the ribs are — because every human body follows the same skeleton.

An HTML document has the same quality. Once you learn the skeleton, you can look at **any** web page's HTML code and immediately understand how it is organized.

The HTML Skeleton — Explained Line by Line

```
<!DOCTYPE html>
<html>
  <head>
    <title>Page Title Here</title>
  </head>
  <body>
    <h1>This is a Heading</h1>
    <p>This is a paragraph.</p>
  </body>
</html>
```

Part	What It Does
<code><!DOCTYPE html></code>	Tells the browser: "This is an HTML5 document." Always the first line.
<code><html></code>	The root element. Everything else goes inside this tag.
<code><head></code>	Contains information about the page — not the content itself.
<code><title></code>	Sets the text that appears in the browser tab . Users see this but not on the page.
<code><body></code>	Contains all the visible content of the page — what users actually see.
<code><h1></code>	A large heading . The most important heading on the page.
<code><p></code>	A paragraph of text.

HTML Tags — The Building Blocks

Definition: HTML tags are special words surrounded by angle brackets `< >` that tell the browser what type of content follows.

There are two types of HTML tags:

Type 1: Paired Tags (Opening + Closing)

Most tags come in pairs. The opening tag starts the element. The closing tag ends it. The closing tag has a forward slash `/`.

```
<p>This is a paragraph.</p>
<h1>This is a heading.</h1>
<strong>This text is bold.</strong>
```

Structure: `<tagname>content goes here</tagname>`

Type 2: Singular Tags (Self-Closing)

Some tags do not wrap around content. They stand alone. They do not need a closing tag.

```

<br>
<hr>
```

Singular Tag	What It Does
<code></code>	Inserts an image.
<code>
</code>	Inserts a line break (like pressing Enter).
<code><hr></code>	Inserts a horizontal line across the page.

Interactive Stop-Point: Grab a Partner

Without looking at a reference, try to answer:

1. Which part of the HTML document contains the page title?
2. Which part contains the visible content?
3. Is `<img>` a paired tag or a singular tag? How do you know?
4. What would happen if you forgot to close a `<p>` tag? Discuss your guess.

Quick Recap

Every HTML page has the same skeleton: `<!DOCTYPE html>`, `<html>`, `<head>`, `<title>`, and `<body>`. Tags are either paired (open + close) or singular (self-closing).

5.5 Creating Content with HTML

Introduction

Now that you know the skeleton, let's fill it with real content. In this section, you will learn how to add headings, paragraphs, links, images, lists, tables, and comments to your web page.

5.5.1 Headings

The Hook

Open any newspaper or textbook. You see a big title at the top. Below it, section headings. Below those, sub-section headings. The size gets smaller as you go deeper. This hierarchy helps you navigate the content.

HTML headings work the same way. They create a clear, organized structure for your page.

The Explanation

HTML has **six levels of headings**, from `<h1>` (the largest and most important) to `<h6>` (the smallest).

```
<h1>Main Title (Largest)</h1>
<h2>Section Heading</h2>
<h3>Sub-section Heading</h3>
<h4>Smaller Sub-heading</h4>
<h5>Even Smaller</h5>
<h6>Smallest Heading</h6>
```

Why headings matter:

Purpose	Explanation
Organization	They break content into clear sections for readers.
SEO	Search engines like Google use headings to understand what your page is about.
Consistency	All browsers display headings with the same default sizes.

Rule of thumb: Use `<h1>` only once per page (the main title). Use `<h2>` for major sections, `<h3>` for sub-sections, and so on.

Example: Headings in a Page

```
<!DOCTYPE html>
<html>
  <head>
    <title>Importance of Headings</title>
  </head>
  <body>
    <h1>Pakistan – My Country</h1>
    <p>Pakistan is a beautiful country in South Asia.</p>

    <h2>Geography</h2>
    <p>Pakistan has mountains, plains, and coastal areas.</p>

    <h3>The Himalayas</h3>
    <p>Some of the world's highest peaks are in Pakistan.</p>
  </body>
</html>
```

Interactive Stop-Point: Pause & Think

You are building a web page about your school. Plan the heading structure:

- What would your `<h1>` be?
- What three things would be `<h2>` sections?
- For one of those sections, what would be an `<h3>` sub-section?

Write it out on paper before typing any code.

Quick Recap

HTML headings `<h1>` to `<h6>` organize your content from most important to least important. Always use `<h1>` for the main title.

5.5.2 Paragraphs

The Explanation

Definition: The `<p>` tag defines a **paragraph** — a block of text with automatic spacing above and below it.

```
<p>This is the first paragraph. It talks about one idea.</p>
<p>This is the second paragraph. It talks about another idea.</p>
```

The browser automatically adds a blank line between paragraphs. You do not need to press Enter multiple times.

Key rule: Every separate idea or block of text should be wrapped in its own `<p>` tag.

Interactive Stop-Point: Pause & Think

What do you think happens if you press Enter inside a `<p>` tag while writing HTML? For example:

```
<p>This is line one.
This is line two.</p>
```

Try it in your browser. Does the second line appear on a new line? Why or why not? What tag would you use to force a line break?

(Hint: look at the `<br>` tag from the previous section.)

Quick Recap

The `<p>` tag wraps a paragraph. The browser adds spacing automatically. Use a new `<p>` tag for each separate block of text.

5.5.3 Links

The Hook

The word "HyperText" in HTML stands for text that contains **links to other text**. Links are what make the web a *web* — an interconnected network of pages. Without links, every page would be an island.

Tim Berners-Lee's greatest invention was not HTML. It was the **hyperlink** — the ability to connect one document to another with a single click.

The Explanation

Links are created using the `<a>` tag (anchor tag). The most important attribute is `href` (Hypertext Reference) — this is the address the link goes to.

Basic link to a website:

```
<a href="https://www.example.com">Visit Example.com</a>
```

What the user sees: **Visit Example.com** (clickable blue underlined text)

What happens when clicked: Browser goes to `https://www.example.com`

Link that opens an email:

```
<a href="mailto:student@school.com">Send Email</a>
```

When clicked, this opens the user's email program with the "To" field already filled.

Link structure explained:

```
<a href="ADDRESS">CLICKABLE TEXT</a>
  ↑           ↑           ↑
anchor tag  destination  what user sees
```

Example: A Page with Links

```
<!DOCTYPE html>
<html>
  <body>
    <h1>Useful Websites</h1>
    <p>Visit <a href="https://www.google.com">Google</a> to search the web.</p>
    <p>Learn coding at <a href="https://www.w3schools.com">W3Schools</a>.</p>
    <p><a href="mailto:teacher@school.edu">Email your teacher</a></p>
  </body>
</html>
```

Interactive Stop-Point: Grab a Partner

Add three links to a web page:

1. A link to your favorite website.
2. A link to your school's website (or make one up).
3. A `mailto:` link to a fictional email address.

Test all three in the browser. Do they work?

Quick Recap

The `<a href="...">` tag creates clickable links. The `href` attribute holds the destination address — a website URL or a `mailto:` email address.

5.5.4 Images

The Explanation

Definition: The `<img>` tag inserts an image into a web page. It is a **singular tag** — it has no closing tag.

```

```

Attribute	What It Does
<code>src</code>	Source — the path or URL of the image file.
<code>alt</code>	Alternative text — shown if the image fails to load; also read by screen readers for visually impaired users.

Using an image from your computer:

```

```

This works if `mycat.jpg` is in the same folder as your HTML file.

Using an image from the internet:

```

```

Why `alt` text matters:

- If the image fails to load, the user sees the description instead.
- Screen readers (used by blind users) read the `alt` text aloud.
- Search engines use `alt` text to understand your images.

Always write meaningful `alt` text.

Example

```
<!DOCTYPE html>
<html>
  <body>
    <h1>My Pet</h1>
    
    <p>This is my cat, Mochi. He loves to sit by the window.</p>
  </body>
</html>
```

Interactive Stop-Point: Pause & Think

Save a photo on your computer in the same folder as your HTML file. Name it `photo.jpg`. Now add it to your web page using the `<img>` tag.

Then try:

1. Change the `src` to a wrong file name (like `wrongname.jpg`). What happens?
2. Remove the `alt` attribute completely. Does anything look different in the browser? Should you still include it?

Quick Recap

The `` tag inserts an image. Always include `alt` text — it is important for accessibility and good practice.

5.5.5 Lists

The Hook

Look at any recipe, instruction manual, or shopping list. The content is organized as a **list** — not one long paragraph. Lists make it easier to read, scan, and follow steps.

HTML gives you two types of lists. You already know them from everyday life.

The Explanation

Type 1: Unordered List (`<ul>`) — Bulleted list. Use when order does not matter.

```
<ul>
  <li>Apples</li>
  <li>Bananas</li>
  <li>Mangoes</li>
</ul>
```

Browser output:

- Apples
 - Bananas
 - Mangoes
-

Type 2: Ordered List (`<ol>`) — Numbered list. Use when order matters.

```
<ol>
  <li>Boil water</li>
  <li>Add tea leaves</li>
  <li>Wait 3 minutes</li>
  <li>Pour into cup</li>
</ol>
```

Browser output:

1. Boil water
2. Add tea leaves
3. Wait 3 minutes
4. Pour into cup

Key tags:

Tag	Name	Purpose
<code></code>	Unordered List	Container for a bulleted list
<code></code>	Ordered List	Container for a numbered list
<code></code>	List Item	A single item inside <code></code> or <code></code>

Example: Lists on a Page

```
<!DOCTYPE html>
<html>
  <body>
    <h2>My Favorite Subjects</h2>
    <ul>
      <li>Computer Science</li>
      <li>Mathematics</li>
      <li>Physics</li>
    </ul>

    <h2>How to Save a File in Notepad</h2>
    <ol>
      <li>Click on "File" in the top menu.</li>
      <li>Click "Save As".</li>
      <li>Choose a folder.</li>
      <li>Type a file name.</li>
      <li>Click "Save".</li>
    </ol>
  </body>
</html>
```

Interactive Stop-Point: Grab a Partner

Create a web page with two lists:

1. An **unordered list** of five things in your school bag.
2. An **ordered list** of the steps you take to get ready for school in the morning.

Which list type makes more sense for each? Why?

Quick Recap

`<ul>` creates a bulleted list. `<ol>` creates a numbered list. Each item goes inside `<li>` tags. Use ordered lists when sequence matters.

5.5.6 Tables

The Hook

Imagine you need to display the marks of 30 students across 5 subjects. If you wrote it as a paragraph, it would be impossible to read. But in a table, it becomes instantly clear — rows for students, columns for subjects.

HTML tables are designed exactly for this: displaying structured data.

The Explanation

A table in HTML is built with three main tags working together:

Tag	Name	What It Does
<code><table></code>	Table	The outer container for the entire table.
<code><tr></code>	Table Row	Defines one horizontal row.
<code><th></code>	Table Header	A header cell — text is bold and centered by default.
<code><td></code>	Table Data	A regular data cell.

Think of it like a grid:

- `<table>` is the whole grid.
 - Each `<tr>` is one row of the grid.
 - Each `<th>` or `<td>` is one cell in that row.
-

Example: A Student Marks Table

```
<!DOCTYPE html>
<html>
  <body>
    <h2>Student Marks</h2>
    <table>
      <tr>
        <th>Name</th>
        <th>Maths</th>
        <th>Science</th>
      </tr>
      <tr>
        <td>Ali</td>
        <td>85</td>
        <td>90</td>
      </tr>
      <tr>
        <td>Sara</td>
        <td>78</td>
        <td>88</td>
      </tr>
      <tr>
        <td>Usman</td>
        <td>92</td>
        <td>76</td>
      </tr>
    </table>
  </body>
</html>
```

How to read this table's code:

- First `<tr>` = the header row (Name, Maths, Science).
- Each following `<tr>` = one student's data.
- `<th>` = bold header text.
- `<td>` = regular data.

Interactive Stop-Point: Pause & Think

Build a table with this data:

Day	Weather	Temperature
Monday	Sunny	35°C
Tuesday	Cloudy	28°C
Wednesday	Rainy	22°C

Write the full HTML code for this table. Include a header row using `<th>`. Test it in your browser.

Quick Recap

HTML tables use `<table>`, `<tr>`, `<th>`, and `<td>` to display data in rows and columns. Use `<th>` for headers and `<td>` for data cells.

5.5.7 HTML Comments

The Explanation

Definition: An HTML comment is a note inside your code that the browser **ignores completely**. Comments are visible in the code but invisible on the web page.

Syntax:

```
<!-- This is a comment. The browser will not display this. -->
```

Everything between `<!--` and `-->` is a comment.

Why use comments?

Use Case	Example
Explain your code	<code><!-- This section displays the navigation menu --></code>
Leave reminders	<code><!-- TODO: Add a contact form here later --></code>
Temporarily hide code	Comment out a section you want to disable for testing.

Example

```
<!DOCTYPE html>
<html>
  <body>
    <!-- This is the main heading of the page -->
    <h1>My School Website</h1>

    <!-- Navigation links will be added below in the next version -->

    <p>Welcome to our school's official web page.</p>

    <!-- Hiding this paragraph temporarily for testing:
    <p>Under construction section here</p>
    -->
  </body>
</html>
```

Interactive Stop-Point: Pause & Think

Add comments to your existing HTML page to:

1. Describe what each section does.
2. Leave a **TODO:** note for something you plan to add later.

Now view the page in the browser. Do your comments appear? They should not.

Quick Recap

HTML comments `<!-- ... -->` are notes in your code that browsers ignore. Use them to explain your code and leave reminders for yourself.

5.6 Styling with CSS

What Is CSS?

Definition: CSS stands for **Cascading Style Sheets**. It is a language used to control the visual appearance of HTML elements — their colors, fonts, sizes, spacing, and layout.

Without CSS, every web page would be plain black text on a white background. CSS transforms that into a designed, visually organized experience.

The Hook

Imagine two identical restaurants. Both serve the same food. One has dim lighting, wooden tables, framed art on the walls, and carefully folded napkins. The other has plastic chairs and fluorescent lights. The food is the same — but your experience is completely different.

CSS is interior design for websites. The HTML content is the same. CSS controls how it **looks** and **feels**.

5.6.1 Basic Structure of CSS

Every CSS rule has the same structure:

```
selector {  
  property: value;  
}
```

Part	What It Is	Example
Selector	Which HTML element to style	<code>h1</code> , <code>p</code> , <code>body</code>
Property	What aspect to change	<code>color</code> , <code>font-size</code> , <code>background-color</code>
Value	What to change it to	<code>red</code> , <code>16px</code> , <code>lightblue</code>

Example — Change the color of all headings to blue:

```
h1 {  
  color: blue;  
}
```

Example — Style multiple properties at once:

```
p {  
  font-family: Arial;  
  font-size: 16px;  
  color: darkgrey;  
}
```

Full CSS Example in an HTML Page

```
<!DOCTYPE html>
<html>
  <head>
    <style>
      body {
        background-color: lightblue;
      }
      h1 {
        color: white;
        text-align: center;
      }
      p {
        font-family: Verdana;
        font-size: 18px;
        color: darkblue;
      }
    </style>
  </head>
  <body>
    <h1>My Styled Web Page</h1>
    <p>This paragraph uses Verdana font, size 18, dark blue color.</p>
  </body>
</html>
```

5.6.2 Three Ways to Add CSS to HTML

The Hook

You can paint a wall in three ways: use a roller on the whole room (efficient, consistent), paint one section by hand (targeted, but tedious), or hire a professional painter who works from a plan (organized, scalable). CSS has the same three approaches.

Method 1: Inline Styles

Inline styles are written directly inside an HTML tag using the `style` attribute.

```
<h1 style="color: blue; text-align: center;">Hello World</h1>
<p style="font-size: 20px;">This paragraph is larger.</p>
```

When to use: Quick, one-time changes on a single element.

Drawback: If you have 50 headings and want to change all their colors, you must edit 50 lines. Hard to maintain.

Method 2: Internal Styles

Internal styles are placed inside a `<style>` tag in the `<head>` section of the HTML document.

```
<head>
  <style>
    h1 {
      color: green;
    }
    p {
      font-size: 16px;
    }
  </style>
</head>
```

When to use: When styling a single page and you want to keep styles organized.

Advantage: One change affects all matching elements on the page.

Method 3: External Stylesheet

An external stylesheet is a separate `.css` file linked to the HTML page.

In your HTML file (`index.html`):

```
<head>
  <link rel="stylesheet" href="styles.css">
</head>
```

In your CSS file (`styles.css`):

```
h1 {
  color: purple;
  text-align: center;
}

p {
  font-family: Arial;
  font-size: 16px;
}
```

When to use: For any real project with multiple pages.

Advantage: One CSS file can style hundreds of HTML pages. Change the CSS once — all pages update automatically.

Comparison Table

Method	Where CSS Goes	Best For
Inline	Inside the HTML tag	Single quick change
Internal	<code><style></code> in <code><head></code>	Styling one page
External	Separate <code>.css</code> file	Multi-page websites

Interactive Stop-Point: Grab a Partner

Create a web page with a heading and two paragraphs. Then:

1. Style the heading using **inline** CSS (turn it red).
2. Style the paragraphs using **internal** CSS (change font size to 18px).
3. Now move all your styles to an **external** CSS file and link it.

Does the page look the same after moving to external? It should. Which method did you find easiest? Which would be best for a 10-page website?

Quick Recap

CSS can be applied three ways: inline (inside the tag), internal (`<style>` in `<head>`), or external (separate `.css` file linked with `<link>`). External is best for real projects.

5.6.3 Styling Fonts, Colors, and Backgrounds

Styling Fonts

CSS gives you complete control over how text looks.

Property	What It Does	Example Value
<code>font-family</code>	Sets the typeface	<code>Arial</code> , <code>Verdana</code> , <code>Times New Roman</code>
<code>font-size</code>	Sets the text size	<code>16px</code> , <code>24px</code> , <code>2em</code>
<code>font-weight</code>	Sets thickness	<code>normal</code> , <code>bold</code>
<code>font-style</code>	Sets italics	<code>normal</code> , <code>italic</code>
<code>color</code>	Sets the text color	<code>red</code> , <code>#333333</code> , <code>rgb(0,0,255)</code>

```
<html>
  <head>
    <style>
      p {
        font-family: Arial, sans-serif;
        font-size: 16px;
        font-weight: bold;
        font-style: italic;
        color: darkblue;
      }
    </style>
  </head>
  <body>
    <p>This text is Arial, 16px, bold, italic, and dark blue.</p>
  </body>
</html>
```

Note: When specifying `font-family`, always include a fallback font.

`Arial, sans-serif` means: "use Arial; if not available, use any sans-serif font."

Styling Colors and Backgrounds

CSS offers three ways to specify colors:

Method	Syntax	Example
Color Name	Plain English	<code>red</code> , <code>blue</code> , <code>lightblue</code>
Hex Code	<code>#RRGGBB</code>	<code>#ff0000</code> (red), <code>#0000ff</code> (blue)
RGB	<code>rgb(R, G, B)</code>	<code>rgb(255, 0, 0)</code> (red)

Background color:

```
body {
  background-color: #f0f0f0;
}

h1 {
  background-color: navy;
  color: white;
}
```

Interactive Stop-Point: Pause & Think

Without running the code, predict what this CSS will do to the page:

```
body {
  background-color: black;
}
h1 {
  color: yellow;
  font-family: Verdana;
}
p {
  color: white;
  font-size: 14px;
}
```

Now run it in your browser. Were you right?

Quick Recap

CSS font properties control typeface, size, weight, and style. Colors can be specified by name, hex code, or RGB values. The `background-color` property sets the background.

5.6.4 Creating Layouts with CSS

The Hook

Every website you visit has a **layout** — a header at the top, navigation on the side or top, main content in the middle, footer at the bottom. This is not accidental. It is designed using CSS layout systems.

In this section you will learn three approaches to creating layouts.

Method 1: Divs and Sections

The `<div>` tag is a generic container. It groups content together so you can style and position it.

The `<section>` tag is similar but carries more meaning (it represents a section of a page).

```
<html>
  <head>
    <style>
      .container { border: 2px solid black; }
      .header    { height: 80px; background-color: lightblue; border: 2px
solid; }
      .content   { height: 200px; background-color: white; border: 2px
solid; }
      .footer    { height: 60px; background-color: lightgrey; border: 2px
solid; }
    </style>
  </head>
  <body>
    <div class="container">
      <section class="header">Header Area</section>
      <section class="content">Main Content Area</section>
      <section class="footer">Footer Area</section>
    </div>
  </body>
</html>
```

The **class** attribute lets you apply CSS to specific elements. A CSS rule starting with `.classname` targets all elements with that class.

Method 2: CSS Grid

CSS Grid is a powerful layout system that arranges elements in **rows and columns**, like a spreadsheet.


```

<html>
  <head>
    <style>
      .grid-container {
        display: grid;
        grid-template-columns: auto auto auto;
        gap: 20px;
        background-color: lightgrey;
        padding: 10px;
      }
      .grid-item {
        background-color: white;
        padding: 20px;
        border: 1px solid;
        text-align: center;
      }
    </style>
  </head>
  <body>
    <div class="grid-container">
      <div class="grid-item">Item 1</div>
      <div class="grid-item">Item 2</div>
      <div class="grid-item">Item 3</div>
      <div class="grid-item">Item 4</div>
      <div class="grid-item">Item 5</div>
      <div class="grid-item">Item 6</div>
    </div>
  </body>
</html>

```

`grid-template-columns: auto auto auto` creates three equal-width columns. Six items fill them: two rows, three columns each.

Method 3: CSS Flexbox

Flexbox is a layout tool that arranges items in a **flexible row or column**. It is great for navigation bars, image galleries, and card layouts.

```

<html>
  <head>
    <style>
      .flex-container {
        display: flex;
        background-color: pink;
        gap: 10px;
        padding: 10px;
      }
      .flex-container > div {
        background-color: lightblue;
        padding: 20px;
        font-size: 24px;
        text-align: center;
        flex: 1;
      }
    </style>
  </head>
  <body>
    <h2>Flexbox Layout</h2>
    <div class="flex-container">
      <div>Box 1</div>
      <div>Box 2</div>
      <div>Box 3</div>
    </div>
  </body>
</html>

```

`display: flex` on the container makes all direct child `<div>` elements line up in a row.

`flex: 1` means each box takes equal space.

Comparison: When to Use Each

Method	Best For
Divs + Sections	Simple page structure (header, content, footer).
CSS Grid	Complex two-dimensional layouts (rows AND columns).
CSS Flexbox	One-directional layouts (a row of cards, a navigation bar).

Interactive Stop-Point: Grab a Partner

Build a page that looks like this using Flexbox:

[Home] [About] [Contact] [Portfolio]

Four navigation links, side by side in a row, each with a background color. Hint: put four `<div>` elements inside a `<div class="flex-container">` and use `display: flex`.

Quick Recap

CSS layouts are created using Divs/Sections for basic structure, CSS Grid for rows-and-columns layouts, and Flexbox for flexible one-directional arrangements.

5.7 Introduction to JavaScript

What Is JavaScript?

Definition: JavaScript is a programming language that runs inside the browser and makes web pages **interactive and dynamic**. It allows web pages to respond to user actions — clicks, keyboard input, mouse movements, and more.

The Hook

In 1995, a programmer named **Brendan Eich** was hired by Netscape (one of the first web browsers). His boss said: "We need a programming language for the web. You have 10 days."

Brendan Eich wrote the first version of JavaScript in **10 days**. He called it Mocha, then LiveScript, then finally JavaScript.

That language — written in 10 days in 1995 — now runs on **virtually every website in the world**. It is one of the most widely used programming languages in history.

And you are about to write your first line of it.

Where Does JavaScript Go in HTML?

JavaScript code is placed inside `<script>` tags. These can go in the `<head>` or at the bottom of the `<body>`.

```

<!DOCTYPE html>
<html>
  <head>
    <title>JavaScript Example</title>
  </head>
  <body>
    <h1>Welcome to JavaScript</h1>
    <script>
      alert("Hello! This is my first JavaScript.");
    </script>
  </body>
</html>

```

When you open this page, a popup box appears with the message. That popup is created by JavaScript.

5.7.1 Variables and Data Types

The Explanation

Definition: A variable is a named container that stores a value. You can store a number, text, or other data in a variable and use it later in your code.

Think of a variable like a **labelled box**. The label is the variable name. The thing inside the box is the value.

Declaring a variable in JavaScript:

```

var name = "Ali";
var age = 15;
var isStudent = true;

```

You can use three keywords to declare variables:

Keyword	When to Use
var	Traditional way. Works everywhere. Used in older code.
let	Modern way. Use when the value might change.
const	Modern way. Use when the value will NEVER change.

JavaScript Data Types:

Type	Example	Description
String	"Hello", "Ali"	Text — always in quotes.
Number	15, 3.14, -7	Any number.
Boolean	true, false	Only two values — true or false.
Undefined	(no value assigned yet)	Variable declared but not given a value.

Example: Variables and Alert

```
<!DOCTYPE html>
<html>
  <body>
    <script>
      var name = "Athar";
      var age = 15;
      alert("Name: " + name + ", Age: " + age);
    </script>
  </body>
</html>
```

Dry Run — Step by Step:

Step	What Happens	State of Variables
1	<code>var name = "Athar"</code> is executed	name = "Athar"
2	<code>var age = 15</code> is executed	name = "Athar", age = 15
3	<code>alert(...)</code> runs	Popup shows: "Name: Athar, Age: 15"

The `+` operator here **joins** (concatenates) strings and variables together into one message.

Interactive Stop-Point: Pause & Think

Without running the code, predict what this will display:

```
var city = "Lahore";
var population = 13000000;
alert("City: " + city + ", Population: " + population);
```

Now run it. Were you right?

Then try: change `var` to `let` for `city` and `const` for `population`. Does it still work?

Quick Recap

Variables store data. Use `var`, `let`, or `const` to declare them. JavaScript has four main data types: String, Number, Boolean, and Undefined.

5.7.2 Functions in JavaScript

The Hook

Imagine you need to greet 30 students in a class. Without functions, you would write `alert("Hello, Student!")` 30 times. That is wasteful and hard to maintain.

A function lets you write the greeting **once** and run it **whenever you need it**. Functions are one of the most powerful ideas in all of programming.

The Explanation

Definition: A function is a reusable block of code that performs a specific task. You define it once and call it as many times as needed.

Basic function syntax:

```
function functionName() {  
  // code to run  
}
```

Calling (running) a function:

```
functionName();
```

Example 1: Simple Function

```
<script>
  function greet() {
    alert("Hello, Student!");
  }

  greet(); // Runs the function – shows the alert
</script>
```

What happens:

1. The function `greet` is **defined** (stored in memory).
 2. `greet()` **calls** the function — JavaScript runs the code inside it.
 3. Alert box appears: "Hello, Student!"
-

Example 2: Function with Parameters

Sometimes you want to pass information **into** a function to customize its behavior.

```
<script>
  function greetStudent(studentName) {
    alert("Hello, " + studentName + "! Welcome to class.");
  }

  greetStudent("Ali");
  greetStudent("Sara");
  greetStudent("Usman");
</script>
```

What happens:

- First call: alert shows "Hello, Ali! Welcome to class."
- Second call: alert shows "Hello, Sara! Welcome to class."
- Third call: alert shows "Hello, Usman! Welcome to class."

Same function — three different results because we passed different values.

Example 3: Function with Multiple Parameters

```
<script>
function addNumbers(a, b) {
  var sum = a + b;
  alert("The sum of " + a + " and " + b + " is: " + sum);
}

addNumbers(5, 3);    // Shows: "The sum of 5 and 3 is: 8"
addNumbers(10, 20); // Shows: "The sum of 10 and 20 is: 30"
</script>
```

Dry Run for `addNumbers(5, 3)`:

Step	Instruction	Value
1	<code>a = 5, b = 3</code>	Parameters received
2	<code>sum = 5 + 3</code>	sum = 8
3	<code>alert(...)</code>	Shows: "The sum of 5 and 3 is: 8"

Class Activity

Task 1: Write a function that takes a student's name as a parameter and displays a personalized greeting. Call it with your own name and two classmates' names.

Task 2: Write a function called `calculateArea` that takes `length` and `width` as parameters. It should calculate the area of a rectangle ($\text{length} \times \text{width}$) and display the result in an alert.

Call it with: `length = 8, width = 5`. What should the output be?

Interactive Stop-Point: Grab a Partner

Write a function called `checkAge` that:

1. Takes `age` as a parameter.
2. If `age` is 18 or above, displays "You can vote."
3. If `age` is below 18, displays "You cannot vote yet."

Call the function three times with ages 15, 18, and 25.

(Hint: You will need an `if...else` statement inside the function.)

Quick Recap

A function is a reusable block of code. Define it once with `function name()`. Call it with `name()`. Use parameters to pass values into the function.

Chapter Summary

In this chapter, you learned the three core technologies of front-end web development: HTML for structure, CSS for styling, and JavaScript for interactivity.

Topic	Key Takeaway
Web Development	The process of building websites using HTML, CSS, and JavaScript.
Front-End	Everything the user sees: HTML, CSS, JavaScript.
Back-End	Servers, databases, and back-end languages (PHP, Python).
Full-Stack	Building both front-end and back-end.
HTML	HyperText Markup Language — the structure of a web page.
HTML Structure	<code><!DOCTYPE html></code> , <code><html></code> , <code><head></code> , <code><title></code> , <code><body></code> .
HTML Tags	Paired (<code><p>...</p></code>) or singular (<code></code> , <code>
</code>).
Headings	<code><h1></code> to <code><h6></code> — organize content hierarchy.
Paragraphs	<code><p></code> — wraps blocks of text.
Links	<code></code> — creates clickable hyperlinks.
Images	<code></code> — inserts images.
Lists	<code></code> for bullets, <code></code> for numbers, <code></code> for items.
Tables	<code><table></code> , <code><tr></code> , <code><th></code> , <code><td></code> — display data in grids.
Comments	<code><!-- ... --></code> — notes ignored by the browser.
CSS	Cascading Style Sheets — controls colors, fonts, and layout.
CSS Methods	Inline, Internal (<code><style></code>), External (<code>.css</code> file).
CSS Fonts	<code>font-family</code> , <code>font-size</code> , <code>font-weight</code> , <code>font-style</code> .
CSS Layouts	Divs/Sections, CSS Grid, CSS Flexbox.
JavaScript	Programming language that makes web pages interactive.
Variables	Named containers for data: <code>var</code> , <code>let</code> , <code>const</code> .
Data Types	String, Number, Boolean, Undefined.
Functions	Reusable blocks of code. Defined once, called many times.
Parameters	Values passed into a function to customize its behavior.

Key Vocabulary

Term	Definition
HTML	HyperText Markup Language — the standard language for creating web pages.
CSS	Cascading Style Sheets — controls the visual appearance of HTML elements.
JavaScript	A programming language that makes web pages interactive.
Tag	An HTML element defined with angle brackets, e.g., <code><p></code> , <code><h1></code> .
Attribute	Extra information added inside an HTML tag, e.g., <code>src</code> , <code>href</code> , <code>alt</code> .
Selector	In CSS, the part that identifies which HTML element to style.
Property	In CSS, the aspect being changed (e.g., <code>color</code> , <code>font-size</code>).
Value	In CSS, what the property is set to (e.g., <code>red</code> , <code>16px</code>).
Variable	A named container that stores a value in JavaScript.
Function	A reusable block of code that performs a specific task.
Parameter	A value passed into a function when calling it.
Alert	A JavaScript popup box that displays a message to the user.
Front-End	The visible part of a website that users see and interact with.
Back-End	The server-side logic, databases, and processes behind a website.
Full-Stack	Development that covers both front-end and back-end.
Inline Style	CSS written directly inside an HTML tag's <code>style</code> attribute.
External Stylesheet	A separate <code>.css</code> file linked to an HTML page.
Flexbox	A CSS layout model for arranging items in a flexible row or column.
Grid	A CSS layout model for arranging items in rows AND columns.
<code><div></code>	A generic HTML container used to group elements for styling.
<code>href</code>	An HTML attribute that specifies the destination of a link.
<code>src</code>	An HTML attribute that specifies the source file for an image.
<code>alt</code>	Alternative text for an image, shown if the image fails to load.

Review Questions

Section 5.1 — Web Development

1. What is web development? Give two reasons why it is a valuable skill.
2. Name the three technologies used in front-end web development and explain what each one does.
3. What is the difference between front-end and back-end development?

Section 5.2 — Components of Web Development

1. What does a web server do?
2. What is a database? Give two examples of data that a database might store.
3. What is a full-stack developer? Why are they valuable?

Section 5.3 — Getting Started with HTML

1. What two tools do you need to start writing HTML? What are examples of each?
2. What file extension must you use when saving an HTML file?
3. Write the HTML code for a page that displays "Hello, World!" as a large heading and "I am learning web development." as a paragraph.

Section 5.4 — HTML Structure

1. What does `<!DOCTYPE html>` tell the browser?
2. What is the difference between the `<head>` and `<body>` sections?
3. What is the difference between a paired tag and a singular tag? Give one example of each.

Section 5.5 — Creating Content

1. What is the largest heading tag? What is the smallest?
2. Write HTML code to create an unordered list of five programming languages.
3. Write HTML code for a link that opens the website `https://www.google.com` with the clickable text "Search Google."
4. What does the `alt` attribute in an `<img>` tag do? Why is it important?
5. Write HTML code for a table with 3 columns (Name, Subject, Marks) and 3 rows of student data.
6. Write an HTML comment that says "This is the navigation section."

Section 5.6 — CSS

1. What does CSS stand for? What is its main purpose?
2. Write the CSS rule to make all paragraphs have a font size of 18px and a color of dark green.
3. What are the three ways to add CSS to an HTML page? Which is best for large projects?

4. Write CSS to create a light grey background for the entire page, with all `<h1>` headings centered and in dark blue.
5. What is the difference between CSS Grid and CSS Flexbox?

Section 5.7 — JavaScript

1. What is JavaScript used for in web development?
2. Write a JavaScript variable that stores the name "Bilal" and display it in an alert.
3. What are the three keywords used to declare variables in JavaScript? What is the difference between `let` and `const`?
4. Write a JavaScript function called `multiply` that takes two numbers as parameters and displays their product in an alert. Call the function with the numbers 6 and 7.
5. Perform a dry run of the following code. What does it display?

```
function greetUser(name) {  
  alert("Welcome, " + name + "! You are logged in.");  
}  
greetUser("Sana");
```

Practical Exercises

Exercise 1 — My Profile Page

Build a complete web page about yourself. It must include:

- Your name as an `<h1>` heading.
- A short paragraph about yourself.
- An unordered list of your three favorite subjects.
- An ordered list of your top five favorite movies or TV shows.
- An image (use any image or a placeholder).
- A link to your favorite website.

Exercise 2 — CSS Styling

Take the profile page from Exercise 1 and add a CSS stylesheet (either internal or external) that:

- Sets the page background to a color of your choice.
- Styles the `<h1>` to be centered, a different color, and uses a specific font.
- Makes the paragraph text 16px in a clean readable font.
- Adds a different background color to the list items.

Exercise 3 — JavaScript Calculator

Create a web page with a JavaScript function called `calculator` that:

1. Takes two numbers and an operation as parameters (`"add"`, `"subtract"`, `"multiply"`, `"divide"`).
2. Performs the correct operation.
3. Displays the result in an alert.

Call the function four times, once for each operation with numbers of your choice.

Exercise 4 — Complete Mini Website

Build a three-section web page using CSS layout (Flexbox or Grid) that includes:

- A **header** section with your name and a navigation bar (Home, About, Contact links).
- A **main content** section with a heading, paragraph, and image.
- A **footer** section with a copyright message.

Style all three sections with different background colors. Use an external CSS file.

End of Unit 5: Web Development with HTML, CSS, and JavaScript

You just built the foundation of the internet. Every website you have ever visited — YouTube, Wikipedia, Instagram — is built with these exact three technologies: HTML, CSS, and JavaScript. You now understand what is happening under the surface. That is not a small thing. That is power.